Multiplayer Game Engine Standardized Feature Plan

Core infrastructure, reusable gameplay packs, and Game Themes
Planning revision v1.1 | November 6, 2026

Guide extension points for a flexible browser multiplayer engine while following the current POC task sequence. Proposed gameplay-pack APIs remain future design examples. Compiled SDK interfaces and a common map loader have been reported implemented through T018.

Layer	Owns	Examples
1. Core engine	Shared multiplayer infrastructure and neutral world services.	Accounts, sessions, transport, entity IDs, validated map contracts.
2. Optional gameplay packs	Reusable mechanics selected by a Game Theme.	Combat, inventory, collection, repairing, puzzles, objectives.
3. Game Theme	A game module supplies rules. Its skin supplies presentation.	Charging cows, burrowing gophers, station repair scenarios.

Game Theme terminology

The app presents one Game Theme selection per session. All players and spectators use that selection. Developers work with its game module, skin, assets and metadata. The engine owns the public contract and default Dots and Squares reference theme.

Design examples

Mad Cow Mayhem and Gopher Madness exercise melee, NPC behavior and traversal. Space Station Outpost, Scrapyard Colony, Frontier Town, Haunted Hotel and Tiny Backyard Kingdoms exercise collection, repair, puzzles and other optional mechanics. Dungeon Salvagers can combine exploration with optional combat.

Revision scope

This revision updates map timing, Game Theme terminology, reported development status, appearance selection and automated ZIP import/export. The feature catalogue and dependency tables are retained on pages 2-5. Page 7 preserves an illustrative future pack API.

Status basis

Implementation status comes from the Copilot completion reports shared in the engine conversation. This document records those reports and has not audited the latest local repository. Ownership and implementation timing are tracked separately.

LAYER 1

Core engine features

Core understands players, entities, actions, and worlds. It should not need to understand cows, pitchforks, gophers, or repair stations.

Feature	Purpose	Used by	Timing
Accounts and identity	Registration, sign-in, persistent player identity.	All	Current POC
Lobby and sessions	Create, list, join, start, end, and clean up; enforce the five-session limit.	All	Current POC
Session isolation	Keep each session world, players, and state independent.	All	Current POC
Authoritative networking	Validate player inputs, advance server simulation, distribute state.	All	Current POC
Spectators and reconnects	Observe sessions and restore permitted participation after disconnects.	All	Existing POC tasks
Entity IDs and lifecycle	Stable IDs within a session; create and remove players, NPCs, objects, and effects.	All	Minimal foundation
World bounds and queries	Positions, boundaries, nearby entities, and blocked regions.	Map-based games	Minimal foundation
Collision and navigation	Movement checks and bounded navigation; game rules choose applicable obstacles.	Map-based games	Existing foundation; extend later
Input/action routing	Send movement and named actions to the active game module.	All	Current SDK foundation
Module and skin loading	Load compatible server rules and client presentation; enforce session selection.	All	Current POC
State/event contracts	Versioned snapshots and events for clients and packs.	All	Minimal foundation
Map loading contract	Common runtime map format, independent of authoring method.	Map-based games	Simple now; richer later
Randomness and clock	Controlled randomness and simulation time.	Generation, spawning, puzzles	Extension point; richer later
Results and persistence	Store accounts/results; accept game-defined scores and outcomes.	All	Existing POC scope
App appearance	Selectable light/dark styling for lobby and surrounding application.	All	Planned frontend work

Three separate concerns: application appearance, game presentation, and game rules. A dark lobby does not change farm artwork or cow behavior.

LAYER 2

Optional reusable gameplay packs

Each row belongs to the gameplay-pack layer. A game declares only the capabilities it needs. These are later features unless already required by an existing POC task.

Pack	Purpose	Games using it
NPC behavior	State machines, targeting, patrols, fleeing, and transitions.	Cows, gophers, ghosts, station workers
Interaction	Validate nearby use, activate, open, and inspect actions.	All proposed games
Vitality and recovery	Health, damage, incapacitation, healing, and revival.	Mad Cow, Gophers, Dungeon; optional hazards elsewhere
Combat	Melee/projectile attacks, hit rules, cooldowns, team targeting.	Mad Cow, Gophers, Dungeon
Resource collection	Gather, transfer, and deposit resource quantities.	Scrapyard, Frontier, Station; optional gopher cleanup
Inventory/equipment	Hold items, enforce capacity, equip tools.	Salvagers, farmers, builders, repair crews
Building	Validate placement and create structures.	Scrapyard, Frontier, farm defenses
Repairing	Restore damaged object condition.	Station, Scrapyard, Mad Cow fences
Crafting	Convert recipe inputs into outputs.	Scrapyard, Frontier, gopher traps
Puzzles/mechanisms	Switches, locks, sequences, linked mechanisms.	Haunted Hotel, military lab, escape games
Objectives/rounds	Goals, progress, success, failure, round transitions.	All proposed games
Scoring	Game-defined point rules and result data.	Collection contests, survival, cooperative repair
Spawning/waves	Schedule entities, groups, and escalating encounters.	Mad Cow, Gophers; nonviolent supply deliveries
Traversal modes	Surface, underground, flying, and related movement rules.	Gophers; potentially ghosts and drones
Map generation	Generate maps from seeds, constraints, reusable pieces.	Farms, dungeons, towns, stations

Nonviolent composition: a repair game can use interaction, inventory, collection, repairing, and objectives with no combat. Objectives can track restoring power without using scores.

Pack dependencies

Capability	Required foundation	Optional integration
NPC behavior	Core entities, clock, spatial queries.	Navigation, combat, traversal modes.
Interaction	Core action routing and entities.	Inventory, puzzles, building, repairing.
Vitality/recovery	Core entities and events.	Combat, environmental hazards, revival objectives.
Combat	Vitality and spatial queries.	Inventory for equipment; scoring and objectives.
Resource collection	Interaction and resource accounting.	Inventory when carrying; deposit objectives.
Inventory/equipment	Core entity/item references.	Combat, crafting, tool requirements.
Building	Interaction, placement/collision checks, entity creation.	Resource costs, crafting, repairing.
Repairing	Interaction and object condition.	Materials, tools, repair objectives.
Crafting	Recipes and atomic input/output transactions.	Inventory/resource stores, workstations, unlocks.
Puzzles	Interaction, mechanism state, and events.	Inventory keys, objective progress.
Objectives/rounds	Core clock and events; selected mechanic packs.	Scoring, waves, puzzle outcomes.
Scoring	Core events and results contract.	Any pack that emits score-worthy events.
Spawning/waves	Core entity creation, spawn markers, clock.	NPC behavior, combat, difficulty rules.
Traversal modes	Core movement and collision queries.	NPC behavior, traversal-specific visual effects.
Map generation	Common map schema, seeded randomness, validation.	Objective placement, navigation checks.

Dependency principles

Repairing does not require combat: a station system can start damaged or deteriorate. Building does not require crafting: a game can provide ready-made structures. Inventory is required only when a mechanic actually stores or equips items.

Resource transactions should be validated together on the server. A successful craft consumes its inputs and produces its output as one operation.

LAYER 3

Theme-specific features

Game / feature	Purpose and ownership	Reused capabilities
Mad Cow: farm and pitchforks	The game supplies barns, fences, farmers, weapons, artwork, and balance.	Maps, inventory, combat, interaction.
Mad Cow: cow charges	The game defines charge warning, rush, recovery, and optional fence damage.	NPC behavior, movement, combat, object condition.
Gophers: burrowing	The game defines tunneling, dirt trails, emergence, attacks, and retreat.	Traversal modes, NPC behavior, events.
Gophers: holes and plugging	The game defines emergence points and how players close them.	Spawning, interaction; resources if plugging has a cost.
Gophers: town and military lab	The game supplies streets, cars, buildings, farms, base, layouts, progression.	Authored/generated maps, collision, objectives.
Station: repair scenario	The game defines broken systems and restoration order.	Repairing, resources, objectives, puzzles.
Scrapyard: recipes/projects	The game defines scrap types, recipes, community construction goals.	Collection, inventory, crafting, building.
Hotel: haunted room puzzles	The game defines clues, room relationships, ghost behavior.	Puzzles, interaction, NPC behavior.
Backyard: territory rules	The game defines food sites, nests, ownership, victory conditions.	Collection, objectives; territory pack only if reuse warrants it.

World detail does not automatically add mechanics

A parked car can begin as a static obstacle. Driving, passengers, fuel, and vehicle damage are separate gameplay features. A building exterior does not automatically imply an accessible interior.

Violent and nonviolent variations

Gopher Madness can emphasize combat or containment: spot tunnels, rescue residents, and plug holes. Space Station can focus entirely on coordinated repairs. Haunted Hotel can reward solving and escaping instead of attacking. The game chooses its rules and required packs.

Authored and generated maps

One validated runtime definition for both sources

Approach	Developer supplies	Current timing
Authored	Placed objects, terrain, obstacles, collision shapes and typed spawn markers.	Common schema and authored fixture reported implemented in T018.
Generated	Seed, generation rules, world size, placement constraints and content palette.	Simple seeded generation reported implemented in T018 and required by the POC.
Hybrid	Authored landmarks combined with generated connections or object placement.	Future theme work when a specific game needs it.

Common object contract

Represent bounds, stable object IDs, type IDs, positions, optional tags/properties, explicit collision geometry and typed spawns. The theme module interprets recognized object properties and capabilities. The skin maps them to graphics. A player-spawn marker identifies a legal location where the server can place an authenticated participant.

Validation and fallback

T018 reports a shared loader for authored and generated definitions, including walkability and spawn-connectivity checks. Generation is capped at four attempts before a validated fallback is loaded. Keep later objective placement reachable and record seeds for reproducible tests.

Gopher Madness example

A military-base lab, town streets and surrounding farms can be authored as a fixed map. Later hybrid generation could vary town blocks or farm plots. The module controls underground movement, surface-barrier handling and valid emergence locations. Terrain and markers supply the spatial data.

Rendering and engine ownership

The skin supplies roads, buildings, cars, props, characters, animation, HUD and audio. The server module controls positions, interactions, visibility permissions and outcomes. A parked car can begin as a static obstacle. Driving and accessible building interiors require their own mechanics.

Access through the public SDK

Core services are available through the game context. Reusable packs are explicit dependencies. Games use public contracts rather than engine internals.

Illustrative only: this API shape is not a claim about existing SDK methods.

```
export const gopherMadness = defineGame({
  id: "gopher-madness",
  version: "1.0.0",
  requires: [
    interactionPack, npcBehaviorPack, vitalityPack,
    combatPack, traversalPack, objectivesPack,
  ],
  map: authoredMap(townMap),

  create(ctx, packs) {
    // Core capability
    const gopher = ctx.entities.spawn({
      kind: "mutant-gopher",
      position: townMap.markers.labExit,
    });

    // Reusable pack capability
    packs.traversal.setMode(gopher.id, "underground");

    // Game-specific behavior
    registerGopherBehavior(ctx, packs, gopher.id);
  },
});
```

A nonviolent repair game can select:

```
requires: [
  interactionPack, inventoryPack,
  resourceCollectionPack, repairingPack, objectivesPack,
]
```

SDK boundaries

- Each session receives independent game and pack state.

- The server validates actions and changes authoritative state.

- Packs publish documented events; objectives and scoring can subscribe.

- Clients render permitted state and events through skins.

- Loading detects missing or incompatible dependencies.

- Saved results include the game version; generated maps may also record a seed.

DELIVERY BOUNDARY

Current foundation and future work

Reported development status through T018

Area	Reported status	Next boundary
Setup, protocol and SDK	T001-T008 completed. Module-defined action envelope supports combat and unlock validation.	Use public interfaces for subsequent mechanics.
Accounts and lobby	T009-T014 reported completed, with a 9-character signup minimum and four-column session list.	Name, User, Players, Registering/Running.
Realtime transport	T015 completed and committed as f78efed.	Wire inputs and state into live simulation.
World and navigation	T016 completed and committed as e7c882a.	Use bounded spatial, collision and navigation services.
Default theme	T017 completed and committed as 43adbfd. Dots and Squares uses the normal SDK loader.	Independent module state per session.
Maps	T018 reported completed with shared authored/generated loading. All 43 tests and workspace checks passed at that checkpoint.	T018 commit status was pending in its completion report.
Movement and gameplay	T019 was the next instructed task. Completion has not been reported.	Continue the existing gameplay task sequence.
Import/export and appearance	Agreed requirements recorded in this revision.	Implement through scheduled feature work.

POC and later packs

Continue proving multiplayer with Dots and Squares, including existing tasks for combat, pickups, progression, respawn, rendering, spectators and results. Keep actions, entity capabilities, HUD data and completion conditions extensible. Extract reusable packs when another Game Theme demonstrates a shared need.

Application appearance

A per-user Appearance selector will offer Light, Dark and System using shared UI tokens for login, lobby, settings and management screens. A session creator selects the Game Theme for that session. These are separate stored selections with distinct ownership.

ZIP export, import and test reports

A simple automatic workflow for the POC

Step	Expected behavior
1. Export	The themes repository exports one complete theme folder as a ZIP with manifest, module, skin, required assets and compatible versions. Example planned command: npm run theme:export -- --theme <theme-id>.
2. Import	In the app, open Game Themes, choose Import ZIP and select the archive. Copying a complete folder into themes/ can use the same import path.
3. Test	Stage the candidate and run a bounded automated smoke session through public APIs and the module lifecycle. Generate a report for every attempt.
4. Register	Passing checks register and enable that exact theme revision in the selectable list. A failed attempt is excluded from the list and provides errors for retry.
5. Fix and retry	Download the report, give it to the designer's AI, apply fixes, export a new ZIP and import again.

Minimum smoke-test scope

Check required files and SDK compatibility, then simulate session creation/start, representative module actions, state/events, a second isolated instance, completion and cleanup. Check that importing leaves core files unchanged. Exercise applicable gameplay and spectator states as those engine APIs become available. Give hung or failed tests an explicit failed result.

Downloadable report

Offer Download Report for success and failure. Provide a readable summary plus structured JSON with theme/engine/SDK versions, tested artifact hash, check names, pass/fail results, file paths, error messages, relevant logs and reproduction inputs or seed. Exclude secrets and real account credentials. The report should let an AI agent identify and reproduce the failure.

Developer and runtime roles

The engine repository owns core code, SDK and default example. The themes repository mirrors the per-theme module/skin/assets structure and supplies export tooling. The automated importer performs promotion after a passing test. No additional manual review gate is required for the agreed POC workflow. Run simulations with temporary test data in a separate test process.

Planned capability

ZIP export/import, downloadable reports and an online upload-and-test workshop remain planned. A later browser editor can extend the workshop. This document does not claim those tools or commands are implemented.